'npm' vs 'npx'
What are they?

`npm` (Node Package Manager) and `npx` (Node Package Executor) are both tools used in the Node.js ecosystem, but they serve different purposes and are used in different contexts.

npm

- **Purpose**: `npm` is the default package **manager** for JavaScript, used to install, share, and manage libraries and packages in Node.js projects.
- **Functionality**: It allows users to add libraries to their projects, manage dependencies, and handle version control of those dependencies.
- **Usage**: You typically use `npm` to install packages globally or locally within a project. For example, running `npm install lodash` will add the lodash library to your project's dependencies.
- **Commands**: Beyond installing packages, `npm` includes commands for managing versions and publishing packages, among other capabilities.

npx

- **Purpose**: `npx` is a package runner tool that simplifies the **execution** of Node.js binaries. It was introduced with npm version 5.2.0.
- **Functionality**: `npx` allows you to run packages without downloading them permanently or globally. It's particularly useful for running packages on a one-off basis or when you want to ensure you're using the latest version of a package without installing it.
- **Usage**: You would use `npx` to execute a package directly from the npm registry without installing it first. For example, `npx create-react-app my-app` will start a new React project using the create-react-app command without you needing to have create-react-app installed globally.
- **Commands**: `npx` can also run packages that are already installed locally in the `node_modules` of a project, which can be useful for executing local scripts or binaries.

In summary, while `npm` is used primarily for package management tasks like installation and dependency control, `npx` is used to execute Node.js binaries and commands, making it easy to use tools and scripts without permanently installing them.